

Plan RAG — Prototype de recherche documentaire

Feuille de route progressive, du premier script Python jusqu'à l'intégration dans le SaaS.

Projet	PROJET RAG LITI
Mis à jour le	13 septembre 2026
Objectif final	Ajouter une brique RAG au SaaS immobilier Django (upload → réponse dans un chat IA).
Objectif actuel	Un prototype indépendant et simple, pour comprendre chaque étape avant toute intégration.

Note sur ce document : aucun nom, montant ou information réelle de locataire n'apparaît dans ce plan. Tous les exemples utilisent des données fictives.

C'est quoi le RAG, en une phrase

RAG = *Retrieval-Augmented Generation*, « génération augmentée par la recherche ».

L'idée : une IA ne connaît pas tes documents privés. Plutôt que de lui « apprendre » tes documents, on fait plus simple — quand tu poses une question, on **cherche** d'abord les passages pertinents dans tes documents, et on les **donne** à l'IA avec ta question. Elle répond à partir de ce qu'on vient de lui fournir.

L'analogie : c'est un examen à livre ouvert. L'IA n'apprend rien par cœur, mais on lui ouvre la bonne page au bon moment.

Vocabulaire minimum

Terme	Traduction simple
Extraction	Sortir le texte brut d'un fichier (PDF, Word...).
Chunk	Un morceau de texte de taille raisonnable. « Tronçon ».
Overlap	Chevauchement : les lignes qu'un chunk reprend du précédent.
Embedding	Transformer un texte en liste de nombres qui capture son <i>sens</i> .
Vecteur	Cette liste de nombres.
Similarité	Mesurer à quel point deux vecteurs (donc deux sens) se ressemblent.
Recherche vectorielle	Trouver les chunks dont le sens ressemble le plus à la question.
Prompt	Le texte complet envoyé à l'IA (contexte + question).

Règles de travail

1. **Une étape à la fois.** On ne commence pas l'étape N+1 tant que N ne fonctionne pas.
2. **Chaque script se lance seul** et affiche son résultat. Si ça casse, on sait où.
3. **Pas de nouvelle bibliothèque** sans en avoir besoin maintenant.
4. **Pas de Django, Docker, base de données** avant l'étape 7. Le prototype tourne en local, avec des fichiers.
5. **On committe après chaque étape réussie.** Git est un filet de sécurité, pas une corvée.

0. Mise en place du projet, Git et protection des données

Cette partie ne contient pas de code. Elle explique **où sont les choses, comment Git fonctionne, et comment mes documents confidentiels restent privés**. C'est la base à comprendre avant tout le reste.

0.1 — La structure du projet, élément par élément

```
PROJET RAG LITI/
├── documents/          les PDF utilisés pour les tests
├── src/                le code source Python
│   ├── extract_pdf.py  étape 1 : lire un PDF
│   ├── clean_text.py   étape 2 : nettoyer le texte
│   ├── chunk_text.py   étape 3 : découper en chunks
│   └── demo_embeddings.py  démo : comprendre les vecteurs
├── .gitignore          ce que Git doit ignorer
├── README.md           présentation du projet
├── requirements.txt     les bibliothèques nécessaires
└── PLAN_RAG.pdf        ce document
```

Élément	À quoi ça sert
documents/	Le dossier qui contient les fichiers PDF servant aux tests. C'est de la donnée , pas du code. Ce dossier reste sur mon ordinateur et n'est jamais envoyé sur GitHub.
src/	« Source » — le dossier où vit tout mon code Python. Le nom <code>src</code> est une convention universelle : n'importe quel développeur qui ouvre mon projet sait immédiatement où regarder. <i>Note : ce dossier s'appelait au départ <code>src (code python)</code> . Il a été renommé en <code>src</code> parce que Python ne sait pas importer un dossier contenant des espaces ou des parenthèses.</i>
extract_pdf.py	Mon premier script. Il ouvre un PDF, en extrait le texte, et l'affiche page par page. L'extension <code>.py</code> signifie « fichier Python ».
requirements.txt	La liste des bibliothèques dont le projet a besoin — une par ligne. Aujourd'hui il contient <code>pypdf</code> . Son intérêt : n'importe qui (ou moi sur un autre ordinateur) peut installer exactement ce qu'il faut avec : <code>python3 -m pip install -r requirements.txt</code>
README.md	Le document de présentation. La première chose qu'on voit en arrivant sur le projet : à quoi il sert, comment le lancer. Le <code>.md</code> signifie Markdown, un format texte simple.
.gitignore	La liste des fichiers et dossiers que Git doit faire semblant de ne pas voir. Détaillé juste en dessous.

Le point commun de tous les fichiers commençant par un point (`.gitignore` , `.env` , `.DS_Store`) : ils sont **cachés**. Le Finder ne les affiche pas par défaut. Dans le Terminal, il faut `ls -a` (le `-a` veut dire *all*) pour les voir.

0.2 — Le fichier `.gitignore`, expliqué

Git surveille tous les fichiers du projet. Mais certains ne doivent **jamais** être enregistrés : soit ils sont inutiles, soit ils sont confidentiels. Le `.gitignore` est la liste de ceux-là.

Ce qu'ignorer veut dire exactement — c'est le point qui prête à confusion :

Un fichier ignoré...	Vrai ou faux
reste sur mon ordinateur	VRAI — il n'est ni supprimé ni déplacé
reste utilisable par mon code	VRAI — mes scripts lisent toujours le PDF normalement
est ajouté aux commits	FAUX — Git l'ignore complètement
part sur GitHub	FAUX — il n'est dans aucun commit, donc rien ne peut partir

Autrement dit : **ignorer ≠ supprimer**. Le fichier vit sa vie sur ma machine, Git fait simplement comme s'il n'existait pas.

Le contenu de mon `.gitignore`, ligne par ligne :

Ligne	Pourquoi elle est là
<code>.DS_Store</code>	Fichier créé automatiquement par macOS dans chaque dossier. Il mémorise la taille de la fenêtre et la position des icônes. Aucun rapport avec mon projet — il encombre juste la liste à chaque <code>git status</code> .
<code>__pycache__/</code>	Dossier de fichiers temporaires Python . Python y range une version pré-compilée de mes scripts pour les relancer plus vite. Il se recrée tout seul, donc le sauvegarder n'a aucun intérêt.
<code>.env</code>	Servira à l'étape 6 à stocker ma clé d'API (l'équivalent d'un mot de passe, souvent payant). Si ce fichier partait sur GitHub, n'importe qui pourrait utiliser ma clé à mes frais. Accident classique et coûteux — d'où la règle posée avant que le fichier existe.
<code>documents/</code>	Contient de vrais documents confidentiels servant aux tests. Ces données n'appartiennent pas au code et ne doivent jamais être publiées.
<code>PLAN_RAG.pdf</code>	Ce document. Ignoré par précaution : un document de travail peut facilement contenir des extraits réels sans qu'on y pense.

La commande qui répond à « pourquoi ce fichier est-il ignoré ? »

```
git check-ignore -v documents/mon-fichier.pdf
```

Elle affiche la ligne exacte du `.gitignore` responsable. Indispensable quand un fichier est ignoré sans qu'on comprenne pourquoi — ou qu'il ne l'est pas alors qu'il devrait l'être.

0.3 — Les quatre commandes Git, et ce qu'elles font vraiment

Git fonctionne par **étapes séparées**. Rien ne se fait tout seul, et c'est volontaire : à chaque étape, je décide.

git init

Démarrer Git dans le projet. Crée un dossier caché `.git/` qui contiendra tout l'historique. Se fait **une seule fois**, au tout début de la vie du projet.

git add

Préparer des fichiers. Je choisis ce que je veux enregistrer. Rien n'est encore sauvegardé — je remplis un carton, je ne l'ai pas encore scellé. Cette zone d'attente s'appelle le *staging*. Bonne habitude : nommer les fichiers (`git add src/mon_script.py`) plutôt que tout prendre avec `git add .` — comme ça je vois ce que j'enregistre.

git commit

Créer une trace dans l'historique. Je scelle le carton et j'écris dessus ce qu'il contient. Une sauvegarde datée à laquelle je pourrai toujours revenir.

`git commit -m "étape 1 : extraction du texte PDF"` — le `-m` introduit le message, qui doit dire **quoi**, pas comment.

git push

Envoyer les commits vers GitHub. Jusqu'ici, tout est resté sur mon ordinateur. C'est cette commande — et elle seule — qui publie.

Le point le plus important de cette page : Git n'envoie jamais rien tout seul.

Un `commit` est **local**. Il vit sur mon disque dur, personne d'autre ne le voit. Seul `git push` envoie réellement quelque chose sur GitHub. Tant que je ne l'ai pas tapé, rien n'a été publié.

Le cycle de travail complet, à répéter à chaque étape du plan :

```
git status      # que s'est-il passé depuis la dernière fois ?
git add ...     # je choisis ce que j'enregistre
git status      # je vérifie AVANT de sceller
git commit -m "... " # je sauvegarde
git push        # (seulement quand je veux publier)
```

Le double `git status` n'est pas une coquetterie : c'est le geste qui empêche de commiter un fichier sensible par inadvertance.

0.4 — Les documents réels ne vont jamais dans Git

Le PDF placé dans `documents/` contient de vraies informations de locataires. Il a un seul rôle : **vérifier en local que mes scripts lisent correctement un PDF réel**. Un vrai document, avec sa vraie complexité de mise en page — ce qu'un fichier d'exemple ne reproduirait pas fidèlement.

Usage autorisé	Test local de l'extraction, sur mon ordinateur.
Interdit	<code>git add</code> , <code>git commit</code> , <code>git push</code> sur ce fichier ou sur <code>documents/</code> .
Pour GitHub	Plus tard : un PDF fictif ou anonymisé — ou aucun PDF du tout.
Dans le futur SaaS	Les vrais contrats seront uploadés depuis l'application et stockés de façon sécurisée (base de données, stockage privé). Ils ne feront jamais partie du code source.

Pourquoi cette règle est stricte : supprimer un fichier de Git ne l'efface pas de l'historique. Un fichier commité puis supprimé reste récupérable dans les anciens commits. Si le dépôt a été publié entre-temps, l'exposition est définitive. **La seule protection fiable est de ne jamais l'ajouter.**

Attention aux fichiers dérivés. La règle ne concerne pas que le PDF d'origine : tout fichier que je crée et qui *cite* des données réelles — notes, exemples, captures d'écran, ce plan — tombe sous la même règle. C'est pour cette raison que `PLAN_RAG.pdf` figure dans le `.gitignore` , et que ce document n'utilise que des exemples fictifs.

Vérification effectuée le 13 septembre 2026

Contrôle	Résultat
Le PDF réel est-il suivi par Git ?	Non
Apparaît-il dans l'historique ?	Non — absent de tous les commits
A-t-il été envoyé sur GitHub ?	Non
Fichiers réellement suivis par Git	<code>.gitignore</code> , <code>README.md</code> , <code>requirements.txt</code> , et les scripts de <code>src/</code>

Point restant à vérifier : le dépôt distant doit être en mode **privé**. À contrôler avant tout `git push` .

Partie 1 — Le prototype (étapes 1 à 6)

Étape 1 — Extraction du texte PDF · FAIT

Objectif	Sortir le texte brut du PDF et l'afficher dans le terminal.
Ce que j'apprends	<code>import</code> , variables, boucle <code>for</code> , appel de fonction, chemins relatifs. Et surtout : l'extraction PDF aplatit la mise en page — les tableaux deviennent des lignes empilées.
Fichiers	<code>src/extract_pdf.py</code> , et un PDF placé dans <code>documents/</code>
Bibliothèque	<code>pypdf</code>
Critère de réussite	<code>python3 src/extract_pdf.py</code> affiche le nombre de pages et le texte de chaque page, sans erreur.
Résultat obtenu	1 page lue, texte extrait intégralement, code de sortie 0. 

Étape 2 — Nettoyage du texte · FAIT

Objectif	Transformer le texte brut en texte propre : supprimer les espaces en trop et les lignes vides multiples.
Ce que j'apprends	<code>.strip()</code> , <code>.split()</code> , <code>.join()</code> , et surtout écrire ma première fonction avec <code>def</code> / <code>return</code> . Plus le <code>if __name__ == "__main__":</code> qui permet à un fichier d'être importé sans s'exécuter.
Fichiers	<code>src/clean_text.py</code>
Bibliothèque	Aucune — Python pur.
Critère de réussite	Plus d'espaces en début/fin de ligne, plus de lignes vides consécutives. Comparaison avant/après affichée.
Résultat obtenu	30 caractères parasites supprimés, comparaison affichée. 
Ce que j'ai appris en plus	<code>nettoyer()</code> ne sait rien des PDF : elle prend du texte, elle rend du texte. Quand les documents viendront de Django au lieu du disque, seule la fonction de lecture changera. Une fonction qui ne dépend de rien est une fonction qu'on ne réécrit jamais.

Étape 3 — Découpage en chunks · FAIT (avec une réserve)

Objectif	Couper le texte propre en morceaux de taille raisonnable, sans casser l'information.
Ce que j'apprends	Les listes Python, la boucle <code>while</code> puis <code>for</code> , le <i>slicing</i> <code>texte[debut:fin]</code> , et le concept d' overlap (chevauchement).
Fichiers	<code>src/chunk_text.py</code>
Bibliothèque	Aucune — Python pur.
Réglages	<code>TAILLE_CHUNK = 400</code> caractères, <code>LIGNES_REPRISES = 2</code>
Critère de réussite	Je lis un chunk au hasard et je le comprends sans son contexte.
Résultat obtenu	3 chunks, les 65 lignes du texte retrouvées intactes. Techniquement propre — mais le critère n'est pas pleinement atteint. Voir la note ci-dessous.

Deux versions successives

	Version 1 — découpage au caractère	Version 2 — découpage à la ligne
Principe	Couper tous les 400 caractères, quoi qu'il arrive	Empiler les lignes jusqu'à approcher 400, puis fermer
Chunks	4	3
Mots coupés	Oui — « ci-dessus » devenait « i-dessus »	Non
Lignes intactes	Non	Oui — 65 sur 65

La version 1 a été écrite volontairement naïve : c'est en voyant ses coupures au milieu des mots qu'on comprend pourquoi la version 2 est nécessaire.



Note de compréhension — le vrai problème des chunks

Au début, je pensais que le problème venait surtout du fait qu'un nouveau chunk reprend seulement les deux dernières lignes du chunk précédent. Cet overlap peut aider à conserver un peu de contexte, mais **ce n'est pas la vraie cause du problème**.

Le vrai problème arrive **avant** le découpage, au moment de l'extraction du PDF.

Le document contient des tableaux. À l'origine, une ligne de tableau relie plusieurs informations ensemble :

Locataire	Local	Date	Montant
Nom X	A-12	27/08	21 000 DA

Mais après extraction avec `pypdf`, le tableau devient une suite de lignes séparées :

Nom X
A-12
27/08
21 000 DA

Ensuite, le système découpe cette suite de lignes en chunks. Si une frontière de chunk tombe juste avant le montant, un chunk peut commencer par :

21 000 DA

Le montant n'est pas coupé, la ligne est intacte, **mais le chunk est difficile à comprendre tout seul** car on ne sait plus à quel locataire ou local il correspond.

L'overlap consiste à répéter quelques lignes entre deux chunks, par exemple :

Chunk 1 : lignes 1 à 20
Chunk 2 : lignes 19 à 38

Cela peut aider si l'information importante est juste avant la frontière. Mais si le nom du locataire est beaucoup plus haut dans le texte, répéter seulement deux lignes ne suffit pas. **L'overlap ne peut pas reconstruire la structure d'un tableau que l'extraction PDF a déjà aplatie.**

Donc, en résumé

- le découpage actuel est **techniquement propre** : aucun mot et aucune ligne ne sont coupés ;
- certains chunks commencent néanmoins par un **montant ou une date isolés** ;
- la cause principale est la **perte de structure du tableau lors de l'extraction PDF**, pas seulement la taille des chunks ou l'overlap.

Ce que je décide pour la suite

Pour le prototype, continuer vers les embeddings et la recherche vectorielle est raisonnable, afin de **mesurer** si cela empêche réellement les bonnes réponses. Si la recherche échoue, il faudra revenir à cette étape et soit :

- **reconstruire les enregistrements du tableau** avec des règles Python (détecter qu'une date suit un nom, etc.) ;
- soit **utiliser un extracteur plus adapté aux tableaux**, comme `pdfplumber` .

Le principe général à retenir : *corriger un problème qu'on n'a pas encore mesuré, c'est du travail perdu*. Et quand une étape donne un mauvais résultat, vérifier d'abord si la faute n'est pas à l'étape précédente.

Étape 4 — Embeddings (transformer le texte en nombres)

Objectif	Convertir chaque chunk en un vecteur : une liste de nombres représentant son sens.
Ce que j'apprends	Ce qu'est un espace vectoriel, intuitivement : deux textes qui parlent de la même chose ont des vecteurs proches. « loyer impayé » et « reste dû » ne partagent aucun mot, mais leurs vecteurs se ressemblent. C'est ce qui rend la recherche <i>sémantique</i> supérieure à Ctrl+F .
Fichiers	<code>src/embed.py</code> (nouveau)
Bibliothèque	À décider ensemble — voir ci-dessous.
Critère de réussite	Le script affiche, pour chaque chunk, la taille du vecteur (ex. 384 ou 1536 nombres) et ses 5 premières valeurs. Aucun vecteur vide.

Décision ouverte — modèle local ou API ?

	Local (ex. sentence-transformers)	API (ex. Voyage AI, OpenAI)
Coût	Gratuit	Payant à l'usage
Internet	Non requis	Requis
Installation	~2 Go à télécharger	Juste une clé
Qualité	Bonne	Excellente
Données	Restent sur ma machine	Sortent vers un service externe

À trancher quand on y sera, en tenant compte du fait que le SaaS gérera des données confidentielles.

Précision : Anthropic ne propose pas d'endpoint d'embeddings — l'API Claude sert à *générer du texte* (étape 6), pas à vectoriser. Pour des embeddings par API, il faut un autre fournisseur (Voyage AI, OpenAI...).

📌 Note de compréhension — ce qu'est vraiment un embedding

L'idée en trois phrases. Un ordinateur ne sait pas comparer des « sens ». Il ne sait comparer que des **nombres**.

Donc on transforme chaque texte en une **liste de nombres** — c'est ça, un vecteur. Puis on mesure si deux listes « pointent dans la même direction ». Plus l'angle entre elles est petit, plus les textes sont proches.

Cette mesure s'appelle la **similarité cosinus**. Elle donne un score entre 0 (rien à voir) et 1 (identique).

La version jouet, codée sans rien installer

Pour transformer un texte en nombres, la méthode la plus bête possible : **compter les mots**.

On liste tous les mots qui existent dans nos phrases — disons [loyer, payé, retard, local] — et chaque phrase devient un décompte :

Phrase	loyer	payé	retard	local
« le loyer est payé »	1	1	0	0
« loyer en retard »	1	0	1	0

Voilà. Deux vecteurs, comparables mathématiquement. C'est exactement le principe des vrais embeddings — texte → liste de nombres → comparaison.

Code de la démo : `src/demo_embeddings.py` (ne fait pas partie du pipeline).

Ce qui marche

Question : « *quel loyer est en retard* »

```
0.707 ##### le loyer du local B est en retard ✅
0.378 ##### le loyer du local A est payé
0.000 les charges de copropriété sont réglées
0.000 le contrat de bail arrive à échéance
```

Le bon document remonte en tête. Logique : la question partage les mots `loyer`, `est`, `en`, `retard` avec lui.

Ce qui échoue — et c'est le plus instructif

Question : « *qui n'a pas payé son bail* »

```
0.267 ##### le loyer du local A est payé      <- LE CONTRAIRE
0.267 ##### le contrat de bail arrive à échéance
0.000 le loyer du local B est en retard      <- LA BONNE RÉPONSE
```

Deux échecs graves :

1. **La bonne réponse obtient 0.000.** « en retard » et « pas payé » veulent dire la même chose — mais ils ne partagent **aucun mot**. Le compteur voit deux textes totalement étrangers.
2. **Le document qui remonte dit l'inverse.** « le loyer est payé » arrive en tête d'une question sur les impayés, uniquement parce que le mot `payé` y apparaît. Le `pas` de « pas payé » est traité comme un mot parmi d'autres, sans effet.

Ce que ça m'apprend

Compter les mots, c'est comparer des **formes**, pas des **sens**. C'est `Ctrl+F` en plus sophistiqué.

Un vrai modèle d'embedding fait la même chose — texte → liste de nombres → cosinus — mais il a été entraîné sur des milliards de phrases. Il a « vu » que *en retard*, *impayé* et *reste dû* apparaissent dans les mêmes contextes. Dans son espace, leurs vecteurs sont voisins **même sans mot commun**.

C'est ça qu'on achète avec les ~2 Go du modèle : pas un algorithme plus malin, mais une **connaissance de la langue** intégrée aux nombres.

	Ma démo (compter les mots)	Vrai modèle d'embedding
Taille du vecteur	21 nombres (un par mot du vocabulaire)	384 nombres, toujours la même taille
Contenu des nombres	Des comptages de mots	Une position dans un espace de sens
« en retard » ≈ « impayé »	Non	Oui
Comparaison	Identique — similarité cosinus dans les deux cas	

À retenir : la fonction `similarite_cosinus()` écrite dans la démo est **la vraie**. Elle ne changera pas à l'étape 5. Seule la façon de *fabriquer* les vecteurs change entre la démo et un vrai modèle.

Étape 5 — Recherche vectorielle

Objectif	Poser une question, la transformer en vecteur, et retrouver les 3 chunks les plus proches.
Ce que j'apprends	La similarité cosinus : une formule qui donne un score entre 0 et 1 mesurant la ressemblance de deux vecteurs. Trier une liste, garder le top N. C'est le cœur du « R » de RAG.
Fichiers	<code>src/search.py</code> (nouveau)
Bibliothèque	Probablement <code>numpy</code> .
Critère de réussite	Je pose une question du type « Qui n'a pas payé ? » et le chunk contenant la ligne concernée remonte dans le top 3, avec son score affiché.
Point d'attention	C'est ici qu'on mesurera la réserve de l'étape 3. Si les mauvais chunks remontent, le problème vient de l'étape 3 ou même de l'étape 1 — pas de l'étape 5.

Étape 6 — Réponse générée

Objectif	Envoyer à une IA les chunks trouvés + ma question, et obtenir une réponse rédigée en français.
Ce que j'apprends	Construire un prompt ; la règle d'or « réponds UNIQUEMENT à partir du contexte fourni, sinon dis que tu ne sais pas » ; gérer une clé d'API sans jamais la mettre dans le code (variable d'environnement + <code>.gitignore</code>).
Fichiers	<code>src/answer.py</code> (nouveau), <code>.env</code> (jamais commité)
Bibliothèque	Le SDK du modèle choisi à l'étape 4.
Critère de réussite	Exemple fictif — Question : « Qui n'a pas payé ? » → Réponse : « Nom X n'a rien versé ce mois-ci ; il reste 21 000 DA dus. » La réponse doit être exacte et tirée du document. Et sur une question hors-sujet, l'IA doit répondre qu'elle ne sait pas au lieu d'inventer .
Point d'attention	Tester volontairement une question piège. Une IA qui invente une réponse est plus dangereuse qu'une IA qui dit « je ne sais pas ».

 **Fin du prototype**

À ce stade, le pipeline RAG complet fonctionne en local, en Python pur, et **je comprends chaque étape**. C'est seulement maintenant qu'on parle d'intégration.

Partie 2 — Vers le SaaS (étapes 7 à 9)

🛑 Ne rien commencer ici avant que les étapes 1 à 6 fonctionnent.

Étape 7 — PostgreSQL + pgvector

Objectif	Ranger les chunks et leurs vecteurs dans une vraie base de données au lieu de fichiers.
Pourquoi c'est nécessaire	En prototype, on recalcule tout à chaque lancement. Avec 500 documents, ce serait insupportable. Une base garde les vecteurs et sait chercher dedans très vite.
Ce que j'apprends	SQL de base, ce qu'est une extension PostgreSQL, <code>pgvector</code> et son type <code>vector</code> , la notion d'index.
Critère de réussite	Je relance le programme, il ne recalcule aucun embedding, et la recherche répond en moins d'une seconde.

Étape 8 — Celery (tâches en arrière-plan)

Objectif	Faire tourner l'extraction + découpage + embeddings en dehors de la requête web.
Pourquoi c'est nécessaire	Traiter un PDF peut prendre 30 secondes. Un utilisateur qui upload ne doit pas voir sa page figée : il doit voir « traitement en cours... ».
Ce que j'apprends	Tâche asynchrone, file d'attente, <i>worker</i> , broker (Redis).
Critère de réussite	J'upload un PDF, la page répond immédiatement, et le document apparaît « prêt » quelques secondes plus tard.

Étape 9 — Intégration Django

Objectif	Brancher le tout dans le SaaS : formulaire d'upload, modèles, chat IA.
Ce que j'apprends	Modèles Django, vues, formulaires, et surtout le cloisonnement par client — chaque utilisateur ne doit jamais voir les documents d'un autre.
Critère de réussite	Depuis le SaaS : j'upload un document, je pose une question dans le chat, j'obtiens une réponse sourcée. Et un compte test ne voit aucun document d'un autre compte.
Point d'attention critique	Sécurité et isolation des données. Ce sont des données financières réelles. Cette étape se fait à deux, pas seule.

Suivi de progression

☑ **Étape 0** — Mise en place, Git et protection des données

☒ **Étape 1** — Extraction PDF

☒ **Étape 2** — Nettoyage du texte

☒ **Étape 3** — Chunks *(avec réserve, voir la note)*

☐ **Étape 4 — Embeddings** ← **prochaine étape**

☐ **Étape 5** — Recherche

☐ **Étape 6** — Réponse générée

☐ **Étape 7** — PostgreSQL + pgvector

☐ **Étape 8** — Celery

☐ **Étape 9** — Django

Rappel : tous les scripts se lancent **depuis la racine du projet**, jamais depuis `src/` .

```
python3 src/chunk_text.py      #  correct
```

Le chemin écrit dans le script (`documents/...`) part du dossier **depuis lequel on tape la commande**, pas du dossier où se trouve le script.